Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters

How should a model spend its inference budget?

Zhenbang Lyu

Heidelberg University
Institute of Computer Science

Snell, Lee, Xu, and Kumar (2024)

You Have 16 More Generations

A model tries to solve a competition math problem and gives the wrong answer. Where should the next 16 generations go?

Right approach, wrong arithmetic

The model sets up the right equation but makes a sign error. A revision could fix it.

Wrong approach

The model assumes a relationship the problem never gives. A fresh attempt could find a better setup.

A verifier can also score partial solutions and decide which ones to develop further.

Three Ways to Spend the Budget

1. **Sample independently.** Generate full solutions in parallel. Use weighted best-of-N to select a final answer.
2. **Revise sequentially.** Generate a new attempt conditioned on earlier ones. Split the budget across multiple chains if exploration also matters.
3. **Search with a verifier.** Use a process reward model (PRM) to score partial solutions. Beam search expands the highest-scoring prefixes. Lookahead first generates extra steps before scoring.

Two places to intervene

Revisions change what the model proposes. Verifier search changes which proposals get more compute.

Snell et al. (2024), Sections 2, 5 and 6.

Difficulty Is Relative to the Model

The study uses PaLM 2-S* on 500 MATH test problems. It fine-tunes models for verification and revision.

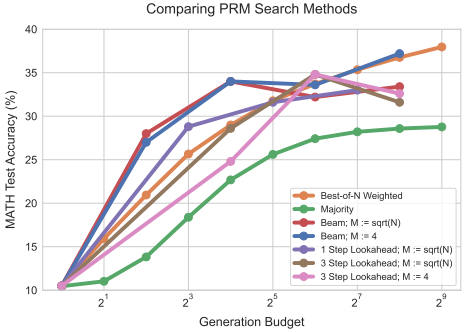
- **Oracle difficulty** Estimate the model's chance of solving each problem from 2,048 attempts. Group the questions into five difficulty bins.
- **Predicted difficulty** Use the verifier's mean score in place of known correctness.
- **Adaptive policy** At each budget, select the best setting for each difficulty bin on one fold and evaluate it on the other. Swap folds and average.

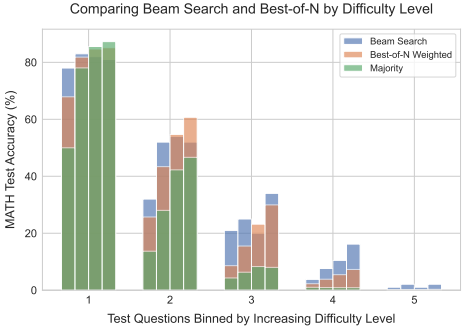
Cost left out of the comparison

The reported generation budgets exclude the cost of estimating difficulty.

Snell et al. (2024), Sections 3–4.

Beam Search Helps When the Budget Is Small

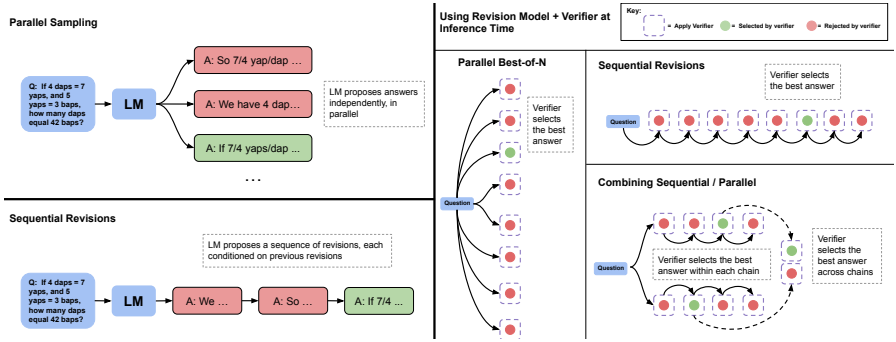




- Beam search leads at low budgets. Lookahead generally trails at the same estimated cost.
- At higher budgets, beam search can hurt in easy bins 1–2, where it can amplify verifier errors. It helps in bins 3–4.
- The hardest bin shows little improvement across methods.

Snell et al. (2024), Section 5.3 and Figure 3.

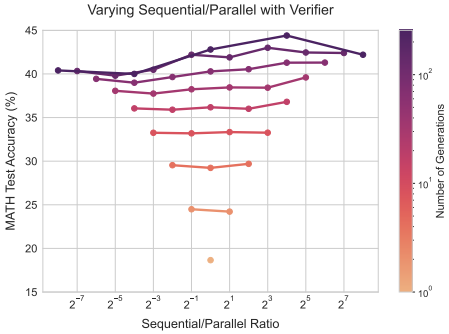
How the Revision Model Uses Earlier Attempts

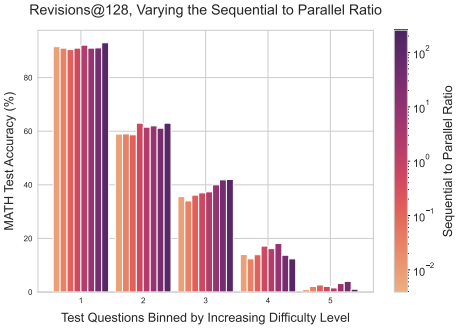


- During training, the model learns to produce a correct solution given up to four related incorrect attempts.
- At test time, a revision can undo a correct solution. The paper selects among attempts with a verifier or majority vote.

Snell et al. (2024), Section 6.1 and Figure 5.

Easy Problems Benefit from Longer Revision Chains

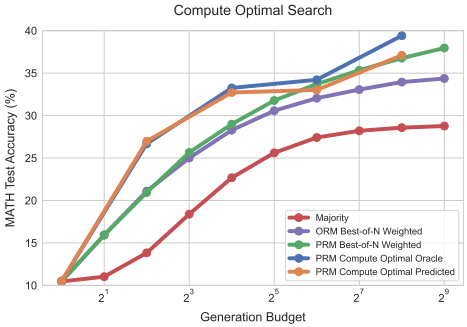


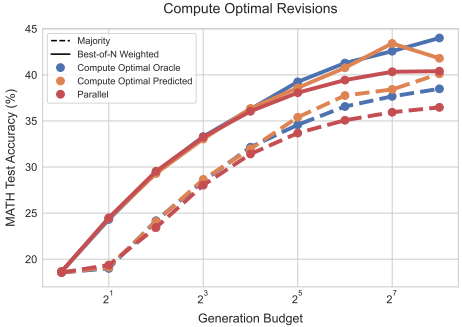


- **Left** As the budget grows, a mix of parallel and sequential compute can beat either extreme.
- **Right** At 128 generations, easy problems favor a single chain. Harder problems often benefit from multiple starting points.

Snell et al. (2024), Section 6.2 and Figure 7.

What Does Difficulty-Based Allocation Buy?





Search 16 generations nearly match 64 with PRM best-of-N.

Revisions 64 generations beat 256 with parallel best-of-N.

Snell et al. (2024), Figures 4 and 8. These are generation-budget comparisons.

Equal Total FLOPs, Two Ways to Spend Them

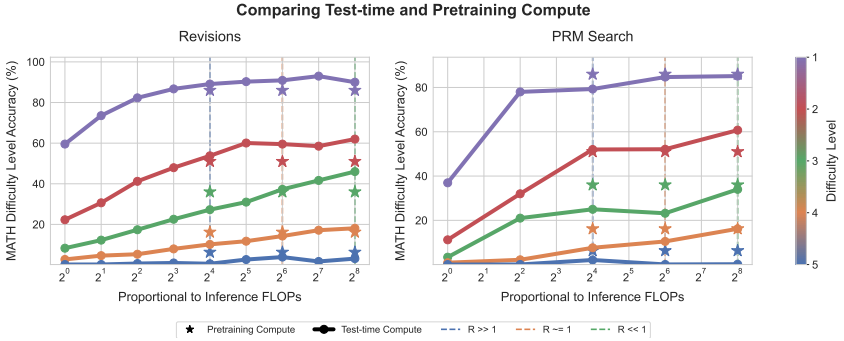
Compare a larger model decoding greedily with a smaller model using adaptive inference. The paper holds pretraining tokens fixed and increases parameter count by about 14×.

$$\underbrace{X + aY}_{\text{smaller model, more inference}} = \underbrace{M(X + Y)}_{\text{larger model, greedy decoding}}$$

- X and Y are the smaller model's baseline pretraining and inference FLOPs. M scales its parameter count and a scales its inference compute.
- R is total inference tokens divided by pretraining tokens. When R is low, the smaller model can spend more compute per inference token at the same total cost.

Snell et al. (2024), Section 7.

Reading the FLOPs Comparison



- **Line** Accuracy as the smaller model spends more on adaptive inference in one difficulty bin.
- **Star** Greedy accuracy of the larger model, placed at the matching FLOPs budget for that workload.
- A line above its matching star favors the smaller model.

Snell et al. (2024), Section 7 and Figure 9.

When Is a Larger Model Worth the Compute?

Extra inference often wins

Easier or intermediate problems, where the smaller model already has some chance of success. A low inference workload makes more extra compute available per request.

A larger model often wins

The hardest problems, where revisions and search make little progress, or a high inference workload, where extra compute per request adds up.

The study tests workload ratios $R = 0.16$, 0.79, and 22.

Snell et al. (2024), Section 7 and Figure 9.

What the Experiment Leaves Open

- **Difficulty at deployment** How much of the inference budget should we spend estimating difficulty before trying to solve the problem?
- **Transfer** The study uses PaLM 2-S* on MATH. Other models and tasks can change which problems are within reach.
- **Combining methods** The paper tests verifier-based selection among revisions. Combining PRM tree search with revision chains remains open.

Snell et al. (2024), Sections 3.2 and 8.

Back to the 16 Generations

- **A promising approach** Spend more of the budget revising it.
- **Several plausible approaches** Try more starting points or use a verifier to guide search.
- **Almost no successful attempts** In these experiments, more generations rarely close the gap.

The practical question

Can we recognize which situation we are in cheaply enough to choose a strategy?

Thank you.

Questions and Discussion